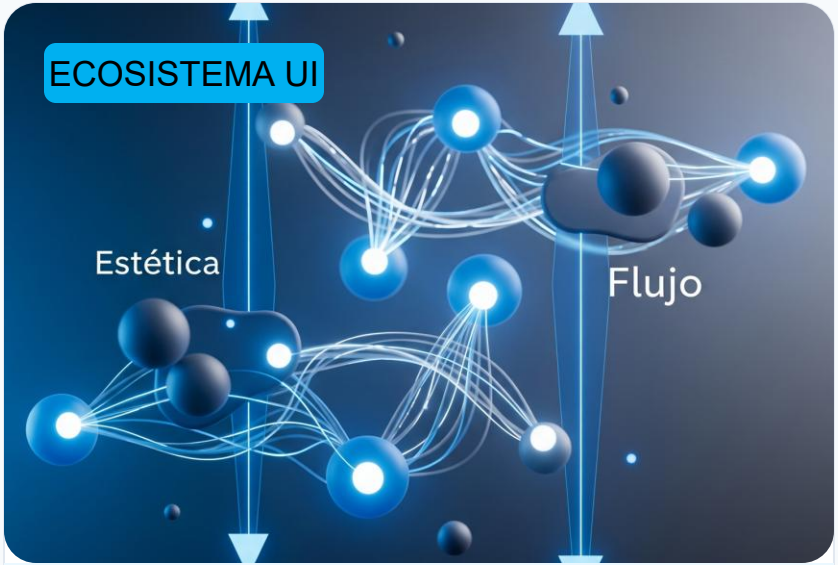
PUCE

Estética y Flujo en Aplicaciones Modernas

Material Design 3 y Navegación Progresiva

La Evolución del Desarrollo Móvil



Convergencia: Experiencia de Usuario (UX)

Las apps modernas no solo deben funcionar: deben **adaptarse y comunicarse** con el usuario.

Estética Semántica

Material Design 3 introduce una visión donde la **UI es un lenguaje**, no una simple pintura decorativa.

Navegación Progresiva

Redefine el flujo: de transacciones frágiles a una **orquestación declarativa** y fuertemente tipada.

Objetivos de Aprendizaje

Material Design 3 & Color

Dominar Material Design 3 y el sistema de Color Dinámico (Material You) para interfaces adaptables.

Gestión de Recursos

Profesionalizar strings.xml, internacionalización y soporte nativo para modo oscuro.

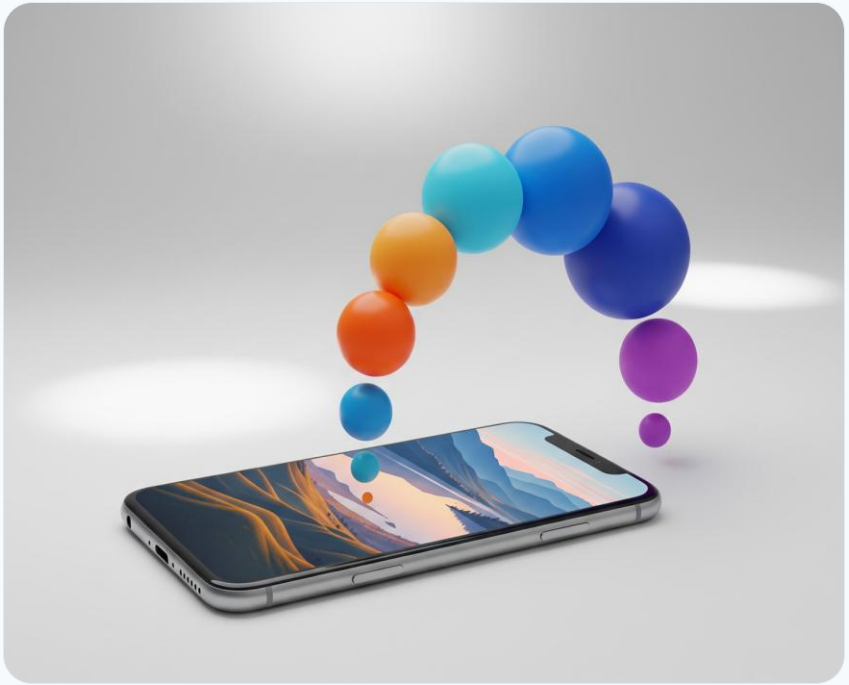
Navegación Avanzada

Construir flujos seguros con NavHost + NavController y navegación fuertemente tipada.

Optimización & UX

Analizar Back Stack de Android vs jerarquía de iOS para optimizar memoria y experiencia.

1.1. Material Design 3: Una Conversación Algorítmica



M3 no es decoración: es comunicación

El **motor Monet** extrae colores del wallpaper para generar paletas armónicas de forma automática.

La app se convierte en un **camaleón digital** que se adapta al entorno visual del usuario.

"Este sistema reduce la fricción emocional al crear una experiencia visual coherente y personal."

1.2. Color Semántico y Accesibilidad

En M3 no se piensa en colores, sino en **funciones**. La accesibilidad es el núcleo del diseño.

PRIMARY

Acciones principales

Componentes clave como FABs y botones de alta prioridad.

SECONDARY

Soporte visual

Menos prominente, utilizado para agrupar o destacar.

TERTIARY

Acentos equilibrantes

Colores de contraste para elementos que necesitan distinción.

SURFACE/BG

Capas neutras

Planos de fondo con elevación por tono en lugar de sombras.

Estándar de Accesibilidad

Para garantizar la legibilidad en diversos contextos visuales, Material Design 3 exige:

- **Contraste mínimo:** 4.5:1 para texto y elementos interactivos.
- **Legibilidad Algorítmica:** Los colores se ajustan para mantener el contraste incluso en temas oscuros.
- **Sin dependencia de matiz:** La jerarquía se basa en la luminancia.

1.3. Evolución del Paradigma de Temas

CARACTERÍSTICA	JAVA / XML	JETPACK COMPOSE
Naturaleza del Tema	Temas **estáticos** definidos en archivos XML rígidos.	Tema como **función reactiva** basada en el estado.
Cambio de Tema	**Recreación de Activity** obligatoria. Parpadeos visuales.	**Recomposición inteligente** sin reiniciar la pantalla.
Material You / Wallpaper	Requiere reinicio manual o lógica compleja de callbacks.	**Actualización automática** inmediata al cambiar el fondo.
Experiencia de Usuario	Interrupción del flujo (Stop/Start).	Transición fluida y continua.

Ejemplo Técnico: SafePassTheme

API 36 / KOTLIN

```kotlin
val colorScheme = when {
  dynamicColor && Build.VERSION.SDK_INT
  >= 31 -> {
    val context = LocalContext.current
    if (darkTheme)
    dynamicDarkColorScheme(context)
    else dynamicLightColorScheme(context)
  }
  darkTheme -> DarkColorScheme
  else -> LightColorScheme
}
```

Implementación Real en API 36

La integración de **Material You** permite que SafePassTheme genere esquemas de color basados en el fondo de pantalla del usuario.

Uso de funciones dinámicas:

- `dynamicDarkColorScheme`: Adapta la UI a tonos oscuros coherentes.
- `dynamicLightColorScheme`: Genera paletas claras vibrantes.

"La personalización algorítmica garantiza armonía visual en todo el sistema."

1.4 Internacionalización (i18n) y Resiliencia Visual

STRINGS.XML

```xml
<resources>
   <string name="welcome">Welcome</string>
   <string name="login">Log In</string>
</resources>
```

El Valor del Recurso Centralizado

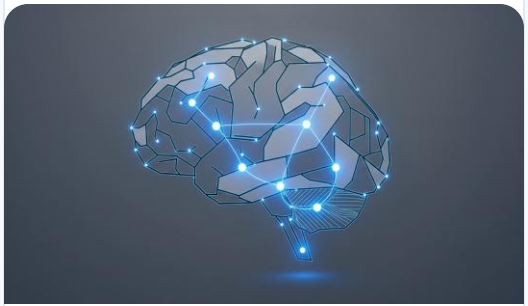
Hardcodear texto es un **pecado capital** del desarrollo móvil. La centralización en **`strings.xml`** es la base de la resiliencia visual.

- **Accesibilidad:** Permite que TalkBack interprete correctamente el contenido textual.
- **Escalabilidad:** Facilita la traducción a múltiples idiomas sin tocar el código fuente.
- **Localización:** Adaptación cultural de formatos, monedas y fechas.

"Un recurso centralizado no es solo una lista de palabras, es la arquitectura que permite que tu App hable al mundo."

2.1. Ecosistema Declarativo de Navegación

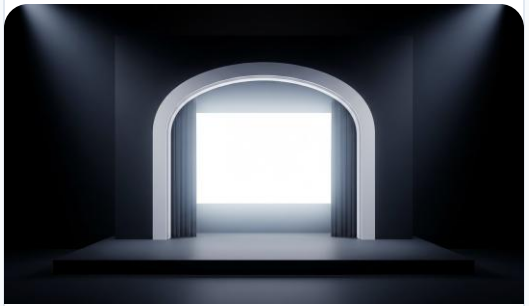
NAVCONTROLLER



Cerebro estratégico

Gestiona el estado de la pila y las transiciones entre pantallas.

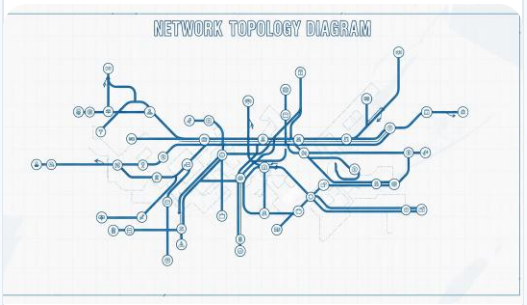
NAVHOST



Escenario dinámico

Contenedor que intercambia los destinos visuales dinámicamente.

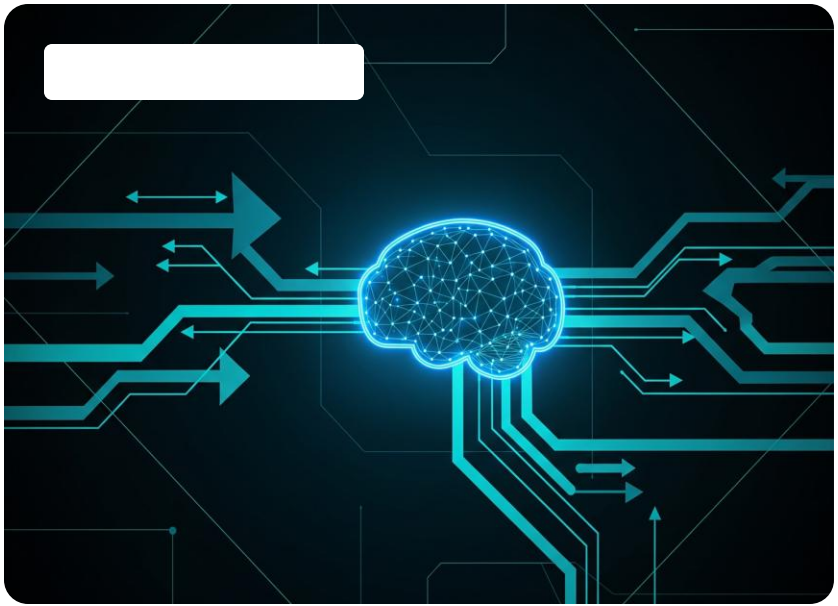
GRAFO



Mapa de destinos

Define la estructura y las conexiones lógicas de toda la navegación.

2.1.1. El NavController: El Guardián del Back Stack



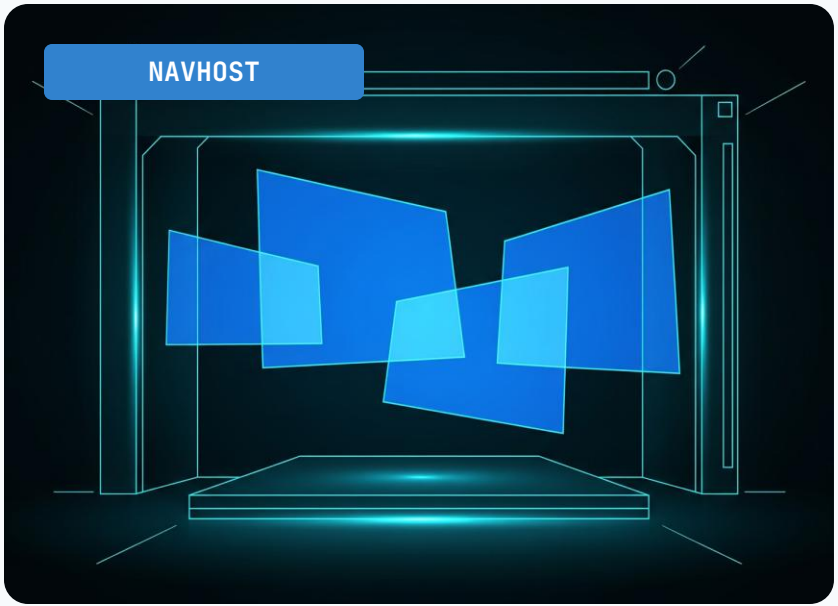
Control Centralizado

- **Gestiona destinos:** Coordina el movimiento entre los diferentes nodos del grafo de navegación.
- **Administra el Back Stack:** Mantiene la pila de pantallas para permitir un retroceso coherente.
- **Transferencia de Datos:** Facilita el paso de argumentos entre pantallas de forma segura.

State Hoisting:

- Debe declararse en niveles altos de la jerarquía para ser compartido por múltiples componentes.

2.1.2. El Escenario de la App: NavHost



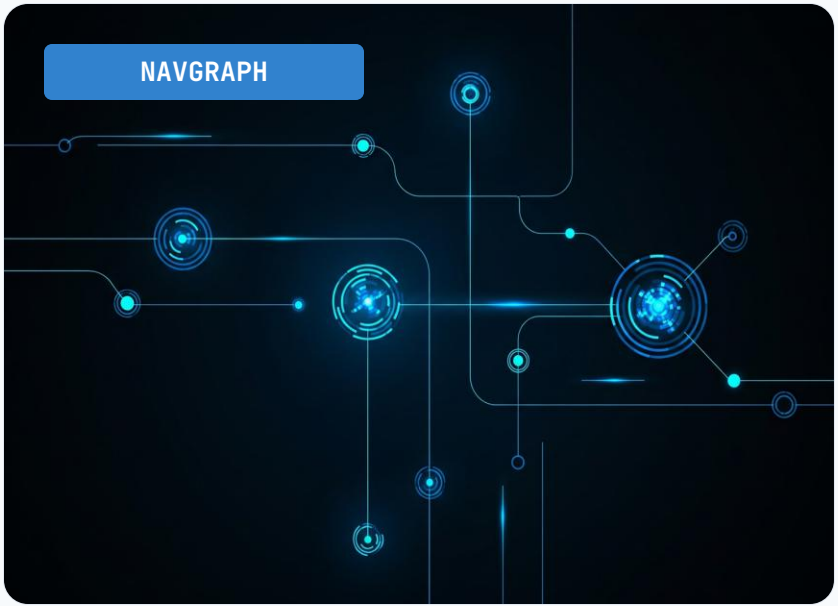
El Contenedor Dinámico

- Funciona como un **escenario teatral** que intercambia Composables según la ruta activa.
- Mantiene la coherencia visual mientras el **NavController** orquesta los cambios.

Error Común:

- **Múltiples NavHost:** Declararlos sin una estrategia clara provoca un **comportamiento errático** del botón "Atrás".

2.1.3. Grafo: El Mapa Declarativo



Grafo de Navegación

- En Compose, el grafo es **código fluido**, no XML rígido.
- Permite una definición programática de rutas y argumentos.
- Facilita la implementación de **Deep Linking** de forma nativa.
- Optimiza la **reconstrucción del flujo** tras cambios de configuración.

2.2. Navegación Tipada (Type-Safety): La Crisis de las Rutas Basadas en Texto



El uso de cadenas de texto para definir rutas es una práctica obsoleta que compromete la estabilidad:

- **Errores silenciosos:** Pequeños typos en un String ("pérfil" vs "perfil") no son detectados por el compilador.
- **Tipos incorrectos:** Pasar un String donde se espera un ID numérico causa **crashes tardíos** en tiempo de ejecución.
- **Deuda técnica:** Aumenta drásticamente el costo de mantenimiento y dificulta el refactorizado seguro del código.

```
composable("home") { HomeScreen() }
composable("profile/{userId}") { ProfileScreen() }
```

```
navController.navigate("perfil/abc")
```

Crash!

2.2.3. API 36: Contratos Tipados

CLASE @SERIALIZABLE

```
@Serializable
data class ProductDetail(
    val productId: Int,
    val title: String
)

navController.navigate(
    ProductDetail(
        productId = 42,
        title = "Teclado Mecánico"
    )
)

@Composable
fun ProductDetailScreen(backStackEntry:
NavBackStackEntry) {
    val args =
backStackEntry.toRoute<ProductDetail>()

    Text("ID: ${args.productId}")
    Text("Producto: ${args.title}")
}
```

El cambio de paradigma hacia contratos tipados garantiza robustez estructural en la navegación:

- **Cada destino es una clase/objeto:** Las rutas dejan de ser Strings mágicos para convertirse en ciudadanos de primera clase.
- **Validación en compilación:** Los errores de navegación se detectan antes de ejecutar la App, no en manos del usuario.
- **Refactorización segura:** Renombrar una ruta o cambiar sus parámetros es verificado por el compilador.
- **Deserialización automática:** El método `toRoute()` extrae los datos de forma segura y sin boilerplate.

Glosario Técnico

Material You
Evolución de Material Design centrada en la personalización dinámica y adaptabilidad del color.

Type-Safety
Garantía de tipos en la navegación que evita errores en tiempo de ejecución al pasar argumentos.

Back Stack
Pila que almacena el historial de destinos visitados para gestionar la navegación hacia atrás.

Recomposición
Proceso de volver a ejecutar funciones Composable cuando sus datos de entrada cambian.

Semantic Color
Uso de nombres lógicos para colores (ej: primary, onSurface) en lugar de valores fijos.

Monet Engine
Motor de extracción de colores del fondo de pantalla para generar paletas personalizadas.

Conclusiones Finales

M3 democratiza el diseño
La interfaz se adapta dinámicamente al usuario mediante Material You.

Navegación Tipada
Elimina errores silenciosos en tiempo de ejecución y fortalece la arquitectura.

Gestión del Back Stack
El manejo experto del historial garantiza un mejor rendimiento y experiencia UX.

